

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	--	------------------------------------

MarmaCitas
Planes de Pruebas Funcionales

Versión: 1.0
Fecha: 14/09/2026

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

HOJA DE CONTROL

Organismo	SENA		
Proyecto	MarmaCitas		
Entregable	Planes de Pruebas Funcionales		
Autor	Maryuri Mosquera		
Versión/Edición	2.0	Fecha Versión	20/08/2026
Aprobado por		Fecha Aprobación	DD/MM/AAAA
		Nº Total de Páginas	19

REGISTRO DE CAMBIOS

Versión doc	Causa del Cambio	Responsable del Cambio	Fecha del Cambio
1.0	Versión inicial	Maryuri Mosquera Andres Mauricio Ramirez Medina	20/08/2026
2.0	Actualización completa según el estado actual del repositorio: se incorporan los módulos de especialidades, servicios, odontólogos, horarios, citas (creación, reprogramación, cancelación, estados, notas, adjuntos) y gestión de pacientes. Se eliminan los casos de prueba obsoletos (cierre de sesión, activación de usuario, no registro) y se agregan los casos correspondientes a las funcionalidades implementadas.	Maryuri Mosquera Andres Mauricio Ramirez Medina	14/09/2026

CONTROL DE DISTRIBUCIÓN

Nombre y Apellidos
Andres Mauricio Ramirez Medina
Maryuri Mosquera

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	--

Tabla de contenido

1	INTRODUCCIÓN	4
1.1	Objetivo.....	4
1.2	Alcance	4
2	TRAZABILIDAD DE CASOS DE PRUEBAS – REQUISITOS.....	5
3	DEFINICIÓN DE LOS CASOS DE PRUEBAS	6
4	ESTRATEGIA DE EJECUCIÓN DE PRUEBAS.....	16
5.	ANEXOS	17
5.1	Limitaciones actuales del sistema (según README del repositorio)	17
5.2	Pruebas automatizadas existentes	17
6.	GLOSARIO	18
7.	BIBLIOGRAFÍA Y REFERENCIAS.....	19

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	------------------------------------

1 INTRODUCCIÓN

1.1 Objetivo

El presente documento tiene como objetivo definir el Plan de Pruebas Funcionales para el sistema MarmaCitas, plataforma web de gestión y programación de citas odontológicas. El propósito es verificar que las funcionalidades implementadas en el repositorio cumplan con los requisitos establecidos y funcionen correctamente.

Mediante la ejecución de los casos de prueba se busca identificar posibles errores en el sistema, comprobar el correcto funcionamiento de sus módulos y garantizar que las acciones realizadas por los usuarios produzcan los resultados esperados.

1.2 Alcance

- El presente Plan de Pruebas Funcionales comprende la verificación de las funcionalidades implementadas en el estado actual del repositorio (commit 6f0485c, 18/09/2026). El sistema está compuesto por un backend en Node.js (Express 5, Mongoose 9, MongoDB Atlas, JWT) y un frontend en React 19 (Vite, Tailwind CSS 4).
- Las pruebas cubren los siguientes módulos:
- **Autenticación y gestión de usuarios:** registro, inicio de sesión, perfil propio, activación/desactivación de usuarios (solo admin), gestión de pacientes (solo recepción/admin).
- **Especialidades:** CRUD completo con Soft Delete, exclusivo para administrador.
- **Servicios odontológicos:** CRUD completo con Soft Delete, asociados a especialidades, exclusivo para administrador.
- **Odontólogos:** CRUD completo con Soft Delete, validación de tarjeta profesional única y especialidad activa, exclusivo para administrador.
- **Horarios laborales:** CRUD completo con Soft Delete, validación de jornada clínica (08:00-12:00 y 14:00-17:00), pausa opcional, un horario activo por odontólogo, exclusivo para administrador.
- **Citas odontológicas:** creación, reprogramación, cancelación, cambio de estado (confirmada, en curso, completada, cancelada, no asistió), notas clínicas, adjuntos, consulta de disponibilidad y reglas de negocio (horario laboral, solapamiento, especialidad, fines de semana, antelación de cancelación).
- **Control de acceso por roles (RBAC):** paciente, odontólogo, recepcionista y administrador.
- Quedan fuera del alcance: pagos en línea (solo existe el campo paymentStatus), recuperación/activación de contraseña, notificaciones, historia clínica global, odontograma y módulo de tratamientos (funcionalidades pendientes según el README del repositorio).

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

2 TRAZABILIDAD DE CASOS DE PRUEBAS – REQUISITOS

Caso de Prueba	RF-001	RF-002	RF-003	RF-004	RF-005	RF-006	RF-007	RF-008	RF-009	RF-010	RF-011	RF-012
CP-001 Registro de usuarios	X											
CP-002 Inicio de sesión		X										
CP-003 Gestión de perfil propio			X									
CP-004 Gestión de especialidades				X								
CP-005 Gestión de servicios					X							
CP-006 Gestión de odontólogos						X						
CP-007 Gestión de horarios							X					
CP-008 Creación de cita								X				
CP-009 Reprogramación de cita									X			
CP-010 Cancelación de cita										X		
CP-011 Cambio de estado y notas de cita											X	
CP-012 Control de acceso por roles												X

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

3 DEFINICIÓN DE LOS CASOS DE PRUEBAS

Registro de usuarios	CP-001	
	No	Si/No
Descripción: Verificar que el sistema permita a un usuario registrarse mediante el formulario de creación de cuenta.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Ingresar a la página de registro de MarmaCitas.		
Pasos: 1. Ingresar a MarmaCitas. 2. Seleccionar la opción de registro. 3. Ingresar el nombre. 4. Ingresar el correo electrónico. 5. Ingresar la contraseña. 6. Seleccionar el botón "Registrarse". 7. Verificar la respuesta del sistema.		
Resultado esperado: El sistema debe procesar la solicitud de registro y, cuando sea exitosa, informar al usuario que la cuenta fue creada y dirigirlo a la página de inicio de sesión.		
Resultado obtenido: En el código de Register.jsx se evidencia que el formulario permite ingresar nombre, correo electrónico y contraseña. Al seleccionar " Registrarse ", se ejecuta la solicitud de registro mediante registerRequest. Si la operación es exitosa, el sistema muestra el mensaje " Usuario creado " y redirige al usuario a la página de inicio de sesión. En caso de error, se muestra un mensaje de error. Por lo tanto, la funcionalidad de registro se encuentra implementada en el Frontend.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Inicio de sesión	CP-002	
	No	Si/No
Descripción: Verificar que un usuario registrado pueda iniciar sesión con correo y contraseña.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con un usuario registrado.		
Pasos: 1. Ingresar a MarmaCitas. 2. Ingresar a la página de login. 3. Ingresar el correo electrónico. 4. Ingresar la contraseña. 5. Seleccionar el botón "Iniciar sesión". 6. Verificar la respuesta del sistema.		
Resultado esperado: El sistema valida las credenciales, genera un token JWT con expiración de 1 día y redirige al usuario según su rol (paciente, odontólogo, recepción o administrador).		
Resultado obtenido: En authController.js, el endpoint POST /api/auth/login busca al usuario por correo; si no existe responde "Usuario no encontrado"; si el usuario está inactivo responde HTTP 403 "El usuario se encuentra inactivo"; si la contraseña no coincide responde "Contraseña incorrecta". En caso de éxito genera un JWT con expiresIn: "1d" y devuelve el token, nombre y rol del usuario. El frontend (Login.jsx + AuthContext) almacena la sesión y redirige según el rol. Por lo tanto, la funcionalidad de autenticación se encuentra implementada.		

Gestión de perfil propio	CP-003	
	No	Si/No
Descripción: Verificar que un usuario autenticado pueda consultar y actualizar su propio perfil.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con un usuario autenticado.		
Pasos: 1. Ingresar a MarmaCitas. 2. Iniciar sesión con las credenciales correspondientes. 3. Ubicar la opción de cierre de sesión.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

4. Seleccionar la opción para cerrar sesión.
5. Verificar el estado de la sesión.

Resultado esperado:

El sistema actualiza la información del perfil y la persiste en la base de datos.

Resultado obtenido:

En userController.js, GET /api/users/me devuelve el perfil del usuario autenticado (sin contraseña, con especialidad poblada si es odontólogo). PUT /api/users/me permite actualizar el nombre; los campos de teléfono, alergias y observaciones médicas solo pueden ser actualizados por pacientes (si otro rol intenta modificarlos, responde HTTP 400). Se validan tipos y longitudes máximas. El frontend incluye páginas de perfil para los cuatro roles (PatientProfile, DoctorProfile, ReceptionProfile, AdminProfile). Por lo tanto, la funcionalidad de gestión de perfil se encuentra implementada.

Gestión de especialidades	CP-004	
	No	Si/No
Descripción: Verificar que el administrador pueda crear, listar, actualizar y desactivar especialidades odontológicas.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con sesión de administrador.		
Pasos: 1. Iniciar sesión como administrador. 2. Acceder al módulo de especialidades (/admin/especialidades). 3. Crear una nueva especialidad. 4. Listar las especialidades activas. 5. Actualizar el nombre de una especialidad. 6. Desactivar una especialidad. 7. Verificar que la especialidad desactivada no aparezca en la lista.		
Resultado esperado: El sistema permite el CRUD completo y las especialidades desactivadas no se listan (Soft Delete).		
Resultado obtenido: En specialtyController.js, el CRUD está implementado con Soft Delete (active: false). Las rutas de lectura (GET /api/specialties) están protegidas por JWT; las rutas de escritura (POST, PUT, DELETE) requieren rol admin. El modelo Specialty valida que el nombre sea único. El frontend incluye AdminSpecialties.jsx. Por lo tanto, la funcionalidad de gestión de especialidades se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Gestión de servicios odontológicos	CP-005	
	No	Si/No
Descripción: Verificar que el administrador pueda crear, listar, actualizar y desactivar servicios odontológicos asociados a una especialidad.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con sesión de administrador. 4. Contar con una especialidad activa existente.		
Pasos: 1. Iniciar sesión como administrador. 2. Acceder al módulo de servicios (/admin/servicios). 3. Crear un servicio con nombre, duración, precio y especialidad. 4. Intentar crear un servicio duplicado en la misma especialidad. 5. Listar los servicios activos. 6. Actualizar un servicio. 7. Desactivar un servicio.		
Resultado esperado: El sistema valida que la especialidad exista y esté activa, que no exista un servicio con el mismo nombre en la misma especialidad, y aplica Soft Delete.		
Resultado obtenido: En serviceController.js, se valida la existencia y estado de la especialidad antes de crear/actualizar; se valida duplicado por nombre dentro de la misma especialidad (HTTP 400). La duración y el precio deben ser mayores que cero (validación del modelo). Las rutas de escritura requieren rol admin. El frontend incluye AdminServices.jsx y AdminCatalog.jsx. Por lo tanto, la funcionalidad de gestión de servicios se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Gestión de odontólogos	CP-006	
	No	Si/No
Descripción: Verificar que el administrador pueda crear, listar, actualizar y desactivar odontólogos.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con sesión de administrador. 4. Contar con una especialidad activa existente.		
Pasos: 1. Iniciar sesión como administrador. 2. Acceder al módulo de odontólogos (/admin/odontologos). 3. Crear un odontólogo con nombre, correo, contraseña, tarjeta profesional, teléfono y especialidad. 4. Intentar crear un odontólogo con tarjeta profesional duplicada. 5. Listar odontólogos activos (con filtro por especialidad). 6. Actualizar un odontólogo. 7. Intentar desactivar un odontólogo con citas futuras o en curso.		
Resultado esperado: El sistema valida la unicidad de la tarjeta profesional, la existencia y estado activo de la especialidad, y bloquea la desactivación si el odontólogo tiene citas confirmadas futuras o en curso.		
Resultado obtenido: En doctorController.js, se valida que el correo y la tarjeta profesional sean únicos (HTTP 409), que la especialidad exista y esté activa (HTTP 404/400). La desactivación verifica citas bloqueantes (in_progress o confirmed futuras) y responde HTTP 409 si existen. Las rutas de escritura requieren rol admin. El frontend incluye AdminDoctors.jsx y ReceptionDoctors.jsx (vista de solo lectura para recepción). Por lo tanto, la funcionalidad de gestión de odontólogos se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Gestión de horarios laborales	CP-007	
	No	Si/No
Descripción: Verificar que el administrador pueda crear, listar, actualizar y desactivar horarios laborales de odontólogos.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con sesión de administrador. Contar con un odontólogo activo existente.		
Pasos: 1. Iniciar sesión como administrador. 2. Acceder al módulo de horarios. 3. Crear un horario para un odontólogo (inicio, fin, pausa opcional). 4. Intentar crear un segundo horario activo para el mismo odontólogo. 5. Intentar crear un horario fuera de la jornada clínica (08:00-12:00 y 14:00-17:00). 6. Intentar crear un horario con hora de inicio mayor o igual a la de fin. 7. Intentar crear un horario con pausa fuera del rango laboral. 8. Desactivar un horario.		
Resultado esperado: El sistema valida que el usuario sea odontólogo y esté activo, que no exista otro horario activo, que las horas tengan formato HH:mm, que la hora de inicio sea menor que la de fin, que la pausa esté dentro del horario, y que todo el horario quepa dentro de la jornada clínica.		
Resultado obtenido: En scheduleController.js, se implementan todas las validaciones descritas. La jornada clínica fija (08:00-12:00 y 14:00-17:00) se define en config/clinicSchedule.js y se valida mediante scheduleFitsClinicHours. El modelo Schedule tiene un índice único parcial { doctor: 1 } con active: true, garantizando un solo horario activo por odontólogo a nivel de base de datos. El frontend incluye AdminCatalog.jsx y DoctorSchedule.jsx (vista del horario propio del odontólogo). Por lo tanto, la funcionalidad de gestión de horarios se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Creación de cita	CP-008	
	No	Si/No
Descripción: Verificar que un paciente, recepcionista o administrador pueda crear una cita odontológica cumpliendo las reglas de negocio.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con usuario autenticado con rol paciente, recepcionista o administrador. 4. Contar con odontólogo activo con horario y servicio activo.		
Pasos: 1. Iniciar sesión como paciente (o recepción/admin). 2. Acceder al módulo de agendamiento. 3. Seleccionar el servicio odontológico. 4. Verificar que solo se muestren odontólogos de la especialidad del servicio. 5. Seleccionar odontólogo, fecha y hora. 6. Confirmar la reserva. 7. Verificar la respuesta del sistema.		
Resultado esperado: El sistema crea la cita en estado confirmed con snapshot del servicio, validando: horario laboral del odontólogo (lunes a viernes, dentro de la jornada clínica), no solapamiento con citas del odontólogo ni del paciente, especialidad del servicio igual a la del odontólogo, y fecha no en el pasado.		
Resultado obtenido: En appointmentController.js, la creación de citas (POST /api/appointments) implementa todas las validaciones descritas. La disponibilidad se calcula en intervalos de 15 minutos. El horario se valida en zona horaria America/Bogota. Se usa control optimista de concurrencia (optimisticConcurrency: true) y transacciones de MongoDB para evitar condiciones de carrera. El frontend incluye BookAppointment.jsx y AppointmentBookingPage.jsx. Por lo tanto, la funcionalidad de creación de citas se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Reprogramación de cita	CP-009	
	No	Si/No
Descripción: Verificar que un paciente, recepcionista o administrador pueda reprogramar una cita existente.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con una cita existente en estado confirmed. 4. Contar con usuario autenticado con rol paciente, recepcionista o administrador.		
Pasos: 1. Iniciar sesión. 2. Acceder a la lista de citas. 3. Seleccionar la opción de reprogramar. 4. Seleccionar nueva fecha y hora. 5. Confirmar la reprogramación. 6. Verificar la respuesta del sistema.		
Resultado esperado: El sistema valida la disponibilidad de la nueva fecha/hora (mismas reglas que la creación) y actualiza dateTime manteniendo el estado confirmed.		
Resultado obtenido: En appointmentController.js, PUT /api/appointments/:id/reschedule implementa la reprogramación con las mismas validaciones de disponibilidad (horario, solapamiento, especialidad, fecha futura). El frontend incluye AppointmentReschedule.jsx. Por lo tanto, la funcionalidad de reprogramación de citas se encuentra implementada.		

Cancelación de cita	CP-010	
	No	Si/No
Descripción: Verificar que un paciente, recepcionista o administrador pueda cancelar una cita, y que el odontólogo no tenga permiso para cancelarla.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con una cita existente. 4. Contar con usuario autenticado.		
Pasos: 1. Iniciar sesión como paciente, recepcionista o administrador. 2. Acceder a la lista de citas. 3. Seleccionar la opción de cancelar.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

4. Confirmar la cancelación.
5. Verificar que la cita quede en estado cancelled.
6. Verificar que el odontólogo no tenga la opción de cancelar.

Resultado esperado:

La cita cambia a estado cancelled y permanece almacenada para trazabilidad. El odontólogo no puede cancelar citas (HTTP 403).

Resultado obtenido:

En appointmentRoutes.js, DELETE /api/appointments/:id está protegida con requireRole("patient", "receptionist", "admin") — el odontólogo recibe 403. La cancelación utiliza Soft Delete lógico (cambio de estado, no eliminación física). Por lo tanto, la funcionalidad de cancelación de citas se encuentra implementada.

Cambio de estado, notas clínicas y adjuntos	CP-011	
	No	Si/No
Descripción: Verificar que el odontólogo pueda cambiar el estado de una cita (en curso, completada, no asistió), registrar notas clínicas y adjuntar archivos.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con una cita asignada al odontólogo autenticado.		
Pasos: 1. Iniciar sesión como odontólogo. 2. Acceder a la lista de citas propias (/odontologo/citas). 3. Iniciar la atención (cambiar a in_progress) 4. Registrar notas clínicas. 5. Adjuntar archivos (PDF, JPG, JPEG, PNG). 6. Completar la cita (cambiar a completed). 7. Marcar una cita como no_show cuando corresponda. 8. Verificar la respuesta del sistema.		
Resultado esperado: El odontólogo puede cambiar el estado de sus citas, registrar notas clínicas y adjuntar archivos. El cambio a in_progress solo es válido cuando la hora de la cita ya llegó (con tolerancia de 15 minutos); no_show solo a partir de la hora exacta.		
Resultado obtenido: En appointmentController.js, PATCH /api/appointments/:id/status (roles doctor, receptionist, admin) y PATCH /api/appointments/:id/notes (solo doctor, citas propias) están implementados. Los adjuntos se suben mediante POST /api/appointments/:id/attachments con Multer, validando tipos MIME permitidos, y se descargan mediante GET /api/appointments/:id/attachments/:attachmentId. El frontend incluye DoctorAppointments.jsx. Por lo tanto, la funcionalidad de cambio de estado, notas y adjuntos se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

Control de acceso por roles (RBAC)	CP-012	
	No	Si/No
Descripción: Verificar que el sistema aplique control de acceso basado en roles en todas las rutas.		
Prerrequisitos 1. Contar con conexión a Internet. 2. Tener acceso a un navegador web. 3. Contar con usuarios de los cuatro roles (paciente, odontólogo, recepcionista, administrador).		
Pasos: 1. Iniciar sesión como paciente. 2. Intentar acceder a rutas de administrador (/admin). 3. Intentar acceder a rutas de recepción (/recepcion). 4. Intentar acceder a rutas de odontólogo (/odontologo). 5. Repetir con cada rol. 6. Verificar las respuestas del sistema.		
Resultado esperado: Cada rol solo puede acceder a sus rutas correspondientes. El backend responde HTTP 403 cuando un rol no autorizado intenta acceder a una ruta protegida.		
Resultado obtenido: En middlewares/authMiddleware.js (verificación de JWT) y middlewares/roleMiddleware.js (verificación de roles), todas las rutas del backend están protegidas. El frontend usa PrivateRoute.jsx con allowedRoles para proteger las rutas por rol. Los accesos no autorizados reciben HTTP 403. Por lo tanto, la funcionalidad de control de acceso por roles se encuentra implementada.		

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	---

4 ESTRATEGIA DE EJECUCIÓN DE PRUEBAS

4.1 Tipos de pruebas

- Pruebas funcionales manuales: ejecución de los casos CP-001 a CP-012 contra la aplicación en ejecución (backend en <http://localhost:5000>, frontend en <http://localhost:5173>).
- Pruebas automatizadas de backend: la suite `node:test` del repositorio ejecuta 148/148 pruebas en 14 suites contra la base `marmacitas_test`. Incluye pruebas de autenticación, especialidades, servicios, odontólogos, horarios, citas, adjuntos y control de acceso.
- Pruebas de frontend: `npm run lint` con 0 errores y `npm run build` exitoso.

4.2 Datos de prueba

El repositorio incluye un script `seed:demo` que crea datos de demostración (administrador, recepción, odontólogo y paciente) con credenciales definidas en `Backend/scripts/seedDemo.js`.

4.3 Entorno de pruebas

- Backend: Node.js ≥ 20.19 , MongoDB (replica set), Express 5, Mongoose 9.
- Frontend: React 19, Vite, Tailwind CSS 4.
- Zona horaria: America/Bogota.

4.4 Criterios de aceptación

- Todos los casos de prueba deben ejecutarse con resultado "Si" (funcionalidad verificada).
- No deben existir errores críticos que impidan el flujo principal de agendamiento de citas.
- Las validaciones de negocio deben responder con mensajes claros en español.

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	------------------------------------

5. ANEXOS

5.1 Limitaciones actuales del sistema (según README del repositorio)

- No hay funcionalidad de pago (solo existe paymentStatus como campo).
- No hay recuperación/activación de contraseña ni envío por correo.
- No hay notificaciones ni recordatorios.
- No existe historia clínica global: el odontólogo ve "Atenciones anteriores" derivadas solo de sus propias citas.
- Sin odontograma, diagnósticos estructurados ni tratamientos.
- Las vistas administrativas de citas usan ventanas de ± 30 días.
- La disponibilidad no anticipa conflictos del paciente (se validan al crear).
- La reprogramación no registra quién la realizó y las notas clínicas no tienen auditoría específica.
- La URL de la API del frontend requiere VITE_API_URL en el entorno.

5.2 Pruebas automatizadas existentes

El repositorio incluye 14 suites de pruebas en Backend/tests/ con 148 pruebas aprobadas, cubriendo: autenticación, roles, especialidades, servicios, odontólogos, horarios, citas (creación, reprogramación, cancelación, estados), adjuntos y validaciones de negocio.

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	--

6. GLOSARIO

Término	Definición
JWT	JSON Web Token: mecanismo de autenticación basado en tokens firmados.
RBAC	Control de acceso basado en roles (Role-Based Access Control).
Soft Delete	Eliminación lógica: el registro se marca como inactivo (<code>active: false</code>) en lugar de eliminarse físicamente.
Snapshot	Copia de la información del servicio (nombre, duración, precio) al momento de crear la cita.
Reprogramación	Cambio de fecha y/o hora de una cita existente.
Jornada clínica	Horario fijo del consultorio: lunes a viernes, 08:00-12:00 y 14:00-17:00.
No show	Estado de la cita cuando el paciente no asistió.

	<Nombre Proyecto> Planes de Pruebas Funcionales	<Unidad Organizativa>
--	---	--

7. BIBLIOGRAFÍA Y REFERENCIAS

- Repositorio del proyecto: <https://github.com/Mauri-Ramirez/MarmaCitas.git>
- README del repositorio (estado actual, stack, pruebas y limitaciones).
- Documentación técnica en docs/: domain-model.md, appointment-rules.md, doctor-rules.md, schedule-rules.md, service-rules.md, testing.md.
- Código fuente: Backend/controllers/, Backend/routes/, Backend/models/, Frontend/src/pages/, Frontend/src/routes/.